DIY Turbo Button Controller - HID Remapper

Created by Liz Clark

https://learn.adafruit.com/diy-turbo-button-controller-hid-remapper

Last updated on 2025-02-19 10:52:01 AM EST

Table of Contents

Overview

Button mashers of the world unite - this project will help to save your thumbs! Use a Feather RP2040 USB Host to listen for a specific button combination from your attached gamepad to trigger "turbo mode" aka rapidly sending A button presses. Otherwise, the Feather acts as a passthrough for your controller, sending all of your gamepad inputs as pressed.

This is a great demo showing how to do "HID Re-mapping" where a keyboard, mouse or joystick has its commands replaced on the fly! It's completely transparent to the host machine, no special drivers, auto-hotkey apps, or other software required.

This hack is inspired by the NES Advantage controller (https://adafru.it/1a8v). Released by Nintendo in 1987 for the NES, it featured two turbo dials that let you dial in the perfect rapid fire A and B button toggling.

Don't Be a Jerk - Be Excellent to Each Other

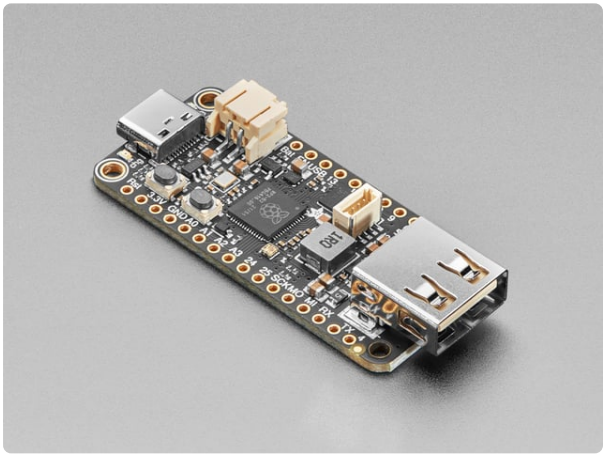
It's never okay to cheat in a game. It's unethical and ruins what should be a fun time for everyone. If that isn't enough of a deterrent, cheating in online play can and will

get you banned by video game companies like Valve. This guide is meant to demonstrate what is possible with USB host and should be used as an experiment or in aiding accessible gaming.

Parts

USB Gamepad

The gamepad used for this project was the Logitech F310 (https://adafru.it/1a8w). You can adjust the code for a different gamepad by adjusting the HID device reports in the code.



Adafruit Feather RP2040 with USB Type A Host

You're probably really used to microcontroller boards with USB, but what about a dev board with two? Two is more than one, so that makes it twice as good! And...

https://www.adafruit.com/product/5723

Snap-on Enclosure for Adafruit Feather RP2040 USB Host

Here is a cute and minimal enclosure for your Adafruit Feather RP2040 with USB Type A Host to keep it safe during use and...

https://www.adafruit.com/product/6057

Pink and Purple Woven USB A to USB C Cable - 1 meter long

This cable is not only super-fashionable, with a woven pink and purple Blinka-like pattern, it's also made for USB C for our modernized breakout boards, Feathers, and...

https://www.adafruit.com/product/5153

Read Your Gamepad Device Report

An HID device report is the data packet sent from the device to a host. In the case of a gamepad, a different device report is sent every time a different button is pressed. For example, the A button has a different packet than the B button.

For this project, you'll need to get your gamepad's device reports to update the code. That way, the code will know when you are pressing an A button or the up button on the D-pad. You'll do this by running some Arduino code that prints out the full device report every time a button is pressed.

```
// SPDX-FileCopyrightText: 2024 Liz Clark for Adafruit Industries
//
// SPDX-License-Identifier: MIT

/*********************************************************************
 Adafruit invests time and resources providing this open source code,
 please support Adafruit and open-source hardware by purchasing
 products from Adafruit!

 MIT license, check LICENSE for more information
 Copyright (c) 2019 Ha Thach for Adafruit Industries
 All text above, and the splash screen below must be included in
 any redistribution
*********************************************************************/

/* This example demonstrates use of both device and host, where
 * - Device runs on native USB controller (roothub port0)
 * - Host depends on MCU:
 *   - rp2040: bit-banging 2 GPIOs with Pico-PIO-USB library (roothub port1)
 *   - samd21/51, nrf52840, esp32: using MAX3421e controller (host shield)
 *
 * Requirements:
 * - For rp2040:
 *   - Pico-PIO-USB library
 *   - 2 consecutive GPIOs: D+ is defined by PIN_USB_HOST_DP, D- = D+ +1
 *   - Provide VBus (5v) and GND for peripheral
 *   - CPU Speed must be either 120 or 240 MHz. Selected via "Menu -> CPU Speed"
 * - For samd21/51, nrf52840, esp32:
 *   - Additional MAX2341e USB Host shield or featherwing is required
 *   - SPI instance, CS pin, INT pin are correctly configured in usbh_helper.h
 */

/* Host example will get device descriptors of attached devices and print it out via
 * device CDC (Serial) as follows:
 *    Device 1: ID 046d:c52f
      Device Descriptor:
        bLength             18
        bDescriptorType     1
        bcdUSB              0200
        bDeviceClass        0
        bDeviceSubClass     0
        bDeviceProtocol     0
        bMaxPacketSize0     8
        idVendor            0x046d
        idProduct           0xc52f
        bcdDevice           2200
        iManufacturer       1     Logitech
        iProduct            2     USB Receiver
        iSerialNumber       0
        bNumConfigurations  1
```

```
 *
 */

// USBHost is defined in usbh_helper.h
#include "usbh_helper.h"
#include "tusb.h"

// Language ID: English
#define LANGUAGE_ID 0x0409

typedef struct {
  tusb_desc_device_t desc_device;
  uint16_t manufacturer[32];
  uint16_t product[48];
  uint16_t serial[16];
  bool mounted;
} dev_info_t;

// CFG_TUH_DEVICE_MAX is defined by tusb_config header
dev_info_t dev_info[CFG_TUH_DEVICE_MAX] = { 0 };

void setup() {
  Serial.begin(115200);

#if defined(CFG_TUH_MAX3421) && CFG_TUH_MAX3421
  // init host stack on controller (rhport) 1
  // For rp2040: this is called in core1's setup1()
  USBHost.begin(1);
#endif

  Serial.println("TinyUSB Dual Device Info Example with HID Report");
}

#if defined(CFG_TUH_MAX3421) && CFG_TUH_MAX3421
//--------------------------------------------------------------------+
// Using Host shield MAX3421E controller
//--------------------------------------------------------------------+
void loop() {
  USBHost.task();
  Serial.flush();
}

#elif defined(ARDUINO_ARCH_RP2040)
//--------------------------------------------------------------------+
// For RP2040 use both core0 for device stack, core1 for host stack
//--------------------------------------------------------------------//

//------------- Core0 -------------//
void loop() {
}

//------------- Core1 -------------//
void setup1() {
  // configure pio-usb: defined in usbh_helper.h
  rp2040_configure_pio_usb();

  // run host stack on controller (rhport) 1
  // Note: For rp2040 pico-pio-usb, calling USBHost.begin() on core1 will have most
of the
  // host bit-banging processing works done in core1 to free up core0 for other
works
  USBHost.begin(1);
}

void loop1() {
  USBHost.task();
  Serial.flush();
}
#endif
```

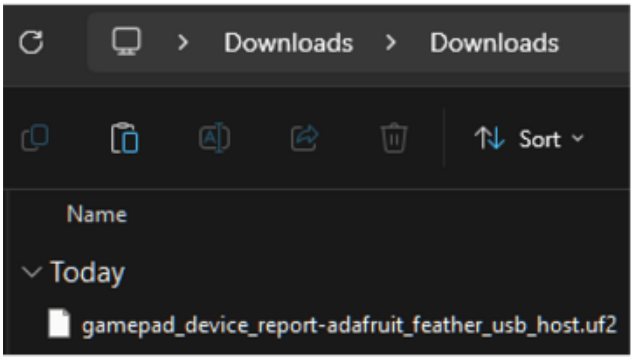
```
//--------------------------------------------------------------------+
// TinyUSB Host callbacks
//--------------------------------------------------------------------+
void print_device_descriptor(tuh_xfer_t *xfer);

void utf16_to_utf8(uint16_t *temp_buf, size_t buf_len);

void print_lsusb(void) {
  bool no_device = true;
  for (uint8_t daddr = 1; daddr < CFG_TUH_DEVICE_MAX + 1; daddr++) {
    // TODO can use tuh_mounted(daddr), but tinyusb has a bug
    // use local connected flag instead
    dev_info_t *dev = &dev_info[daddr - 1];
    if (dev->mounted) {
      Serial.printf("Device %u: ID %04x:%04x %s %s\r\n", daddr,
                    dev->desc_device.idVendor, dev->desc_device.idProduct,
                    (char *) dev->manufacturer, (char *) dev->product);

      no_device = false;
    }
  }

  if (no_device) {
    Serial.println("No device connected (except hub)");
  }
}

// Invoked when device is mounted (configured)
void tuh_mount_cb(uint8_t daddr) {
  Serial.printf("Device attached, address = %d\r\n", daddr);

  dev_info_t *dev = &dev_info[daddr - 1];
  dev->mounted = true;

  // Get Device Descriptor
  tuh_descriptor_get_device(daddr, &dev->desc_device, 18, print_device_descriptor,
0);
}

/// Invoked when device is unmounted (bus reset/unplugged)
void tuh_umount_cb(uint8_t daddr) {
  Serial.printf("Device removed, address = %d\r\n", daddr);
  dev_info_t *dev = &dev_info[daddr - 1];
  dev->mounted = false;

  // print device summary
  print_lsusb();
}

void print_device_descriptor(tuh_xfer_t *xfer) {
  if (XFER_RESULT_SUCCESS != xfer->result) {
    Serial.printf("Failed to get device descriptor\r\n");
    return;
  }

  uint8_t const daddr = xfer->daddr;
  dev_info_t *dev = &dev_info[daddr - 1];
  tusb_desc_device_t *desc = &dev->desc_device;

  Serial.printf("Device %u: ID %04x:%04x\r\n", daddr, desc->idVendor, desc-
>idProduct);
  Serial.printf("Device Descriptor:\r\n");
  Serial.printf("  bLength             %u\r\n"     , desc->bLength);
  Serial.printf("  bDescriptorType     %u\r\n"     , desc->bDescriptorType);
  Serial.printf("  bcdUSB              %04x\r\n"   , desc->bcdUSB);
  Serial.printf("  bDeviceClass        %u\r\n"     , desc->bDeviceClass);
  Serial.printf("  bDeviceSubClass     %u\r\n"     , desc->bDeviceSubClass);
  Serial.printf("  bDeviceProtocol     %u\r\n"     , desc->bDeviceProtocol);
```

```
  Serial.printf("  bMaxPacketSize0     %u\r\n"     , desc->bMaxPacketSize0);
  Serial.printf("  idVendor            0x%04x\r\n" , desc->idVendor);
  Serial.printf("  idProduct           0x%04x\r\n" , desc->idProduct);
  Serial.printf("  bcdDevice           %04x\r\n"   , desc->bcdDevice);

  // Get String descriptor using Sync API
  Serial.printf("  iManufacturer       %u     ", desc->iManufacturer);
  if (XFER_RESULT_SUCCESS ==
      tuh_descriptor_get_manufacturer_string_sync(daddr, LANGUAGE_ID, dev-
>manufacturer, sizeof(dev->manufacturer))) {
    utf16_to_utf8(dev->manufacturer, sizeof(dev->manufacturer));
    Serial.printf((char *) dev->manufacturer);
  }
  Serial.printf("\r\n");

  Serial.printf("  iProduct            %u     ", desc->iProduct);
  if (XFER_RESULT_SUCCESS ==
      tuh_descriptor_get_product_string_sync(daddr, LANGUAGE_ID, dev->product,
sizeof(dev->product))) {
    utf16_to_utf8(dev->product, sizeof(dev->product));
    Serial.printf((char *) dev->product);
  }
  Serial.printf("\r\n");

  Serial.printf("  iSerialNumber       %u     ", desc->iSerialNumber);
  if (XFER_RESULT_SUCCESS ==
      tuh_descriptor_get_serial_string_sync(daddr, LANGUAGE_ID, dev->serial,
sizeof(dev->serial))) {
    utf16_to_utf8(dev->serial, sizeof(dev->serial));
    Serial.printf((char *) dev->serial);
  }
  Serial.printf("\r\n");

  Serial.printf("  bNumConfigurations  %u\r\n", desc->bNumConfigurations);

  // print device summary
  print_lsusb();
}

//--------------------------------------------------------------------+
// String Descriptor Helper
//--------------------------------------------------------------------+

static void _convert_utf16le_to_utf8(const uint16_t *utf16, size_t utf16_len,
uint8_t *utf8, size_t utf8_len) {
  // TODO: Check for runover.
  (void) utf8_len;
  // Get the UTF-16 length out of the data itself.

  for (size_t i = 0; i < utf16_len; i++) {
    uint16_t chr = utf16[i];
    if (chr < 0x80) {
      *utf8++ = chr & 0xff;
    } else if (chr < 0x800) {
      *utf8++ = (uint8_t) (0xC0 | (chr >> 6 & 0x1F));
      *utf8++ = (uint8_t) (0x80 | (chr >> 0 & 0x3F));
    } else {
      // TODO: Verify surrogate.
      *utf8++ = (uint8_t) (0xE0 | (chr >> 12 & 0x0F));
      *utf8++ = (uint8_t) (0x80 | (chr >> 6 & 0x3F));
      *utf8++ = (uint8_t) (0x80 | (chr >> 0 & 0x3F));
    }
    // TODO: Handle UTF-16 code points that take two entries.
  }
}

// Count how many bytes a utf-16-le encoded string will take in utf-8.
static int _count_utf8_bytes(const uint16_t *buf, size_t len) {
  size_t total_bytes = 0;
```

```
  for (size_t i = 0; i < len; i++) {
    uint16_t chr = buf[i];
    if (chr < 0x80) {
      total_bytes += 1;
    } else if (chr < 0x800) {
      total_bytes += 2;
    } else {
      total_bytes += 3;
    }
    // TODO: Handle UTF-16 code points that take two entries.
  }
  return total_bytes;
}

void utf16_to_utf8(uint16_t *temp_buf, size_t buf_len) {
  size_t utf16_len = ((temp_buf[0] & 0xff) - 2) / sizeof(uint16_t);
  size_t utf8_len = _count_utf8_bytes(temp_buf + 1, utf16_len);

  _convert_utf16le_to_utf8(temp_buf + 1, utf16_len, (uint8_t *) temp_buf, buf_len);
  ((uint8_t *) temp_buf)[utf8_len] = '\0';
}

//--------------------------------------------------------------------+
// HID Host Callback Functions
//--------------------------------------------------------------------+

void tuh_hid_mount_cb(uint8_t dev_addr, uint8_t instance, uint8_t const*
desc_report, uint16_t desc_len)
{
  Serial.printf("HID device mounted (address %d, instance %d)\n", dev_addr,
instance);

  // Start receiving HID reports
  if (!tuh_hid_receive_report(dev_addr, instance))
  {
    Serial.printf("Error: cannot request to receive report\n");
  }
}

void tuh_hid_umount_cb(uint8_t dev_addr, uint8_t instance)
{
  Serial.printf("HID device unmounted (address %d, instance %d)\n", dev_addr,
instance);
}

void tuh_hid_report_received_cb(uint8_t dev_addr, uint8_t instance, uint8_t const*
report, uint16_t len)
{
  Serial.printf("Received HID report from device %d instance %d: ", dev_addr,
instance);
  for (uint16_t i = 0; i < len; i++)
  {
    Serial.printf("%02X ", report[i]);
  }
  Serial.printf("\n");

  // Continue to receive the next report
  if (!tuh_hid_receive_report(dev_addr, instance))
  {
    Serial.printf("Error: cannot request to receive report\n");
  }
}
```

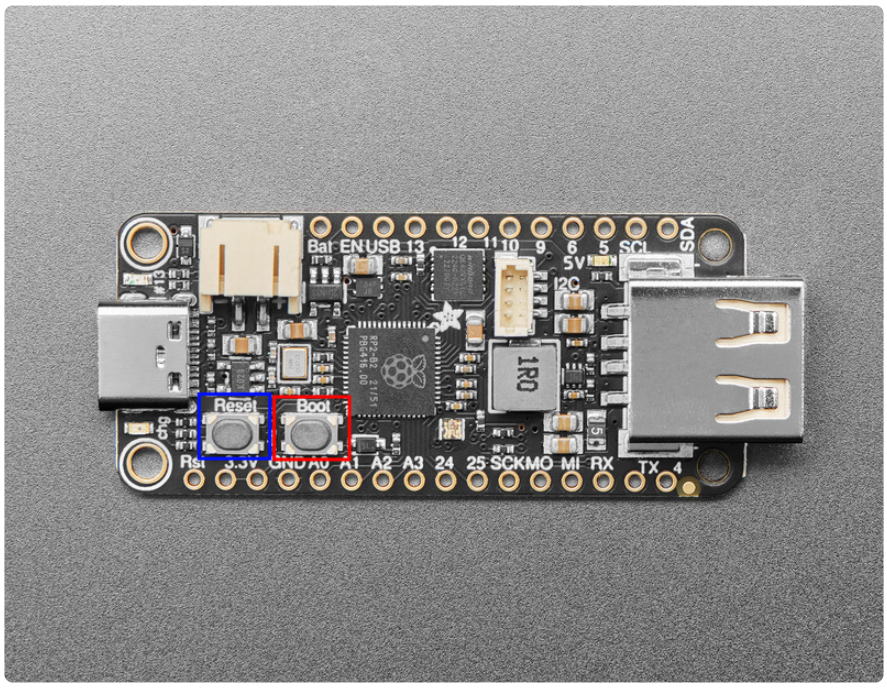
Device Report UF2

The Arduino code is available as a pre-compiled UF2. You can drag and drop the UF2 file onto your Feather RP2040 USB Host board.



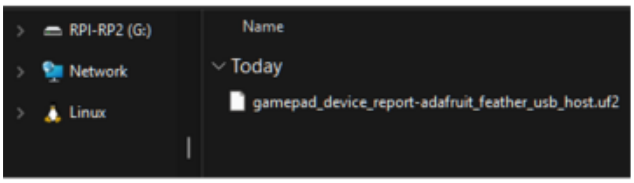
Click the link above to download the UF2 file.

Save it wherever is convenient for you.



Plug your board into your computer, using a known-good data-sync USB cable, directly, or via an adapter if needed.

Hold down the **BOOT/BOOTSEL button** (highlighted in red above), and while continuing to hold it (don't let go!), press and release the **Reset button** (highlighted in blue above). **Continue to hold the BOOT/BOOTSEL button until the RPI-RP2 drive appears!**

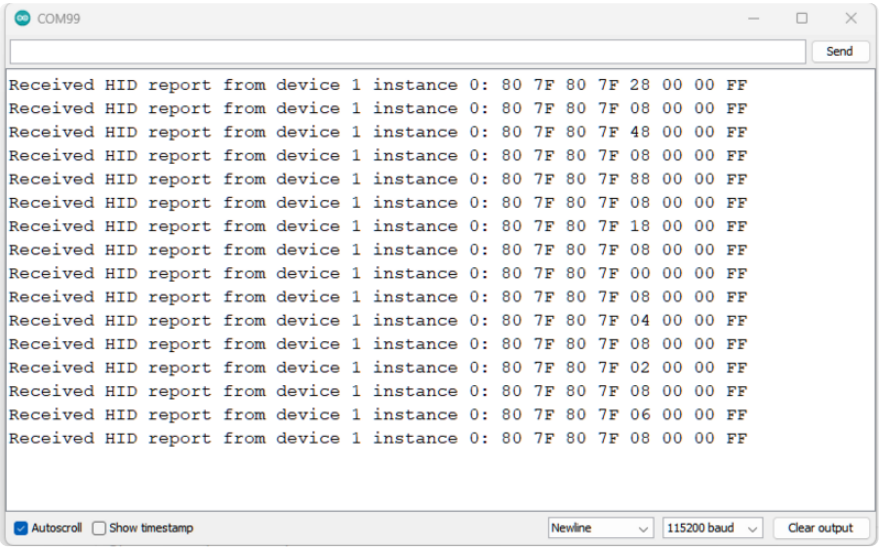


Drag the **UF2** file to **RPI-RP2**.

Reading Reports

Head over to the Arduino IDE and open up the Serial Monitor (**Tools -> Serial Monitor**) at 115200 baud. As you press each button on your gamepad, you'll see a different HID report print to the Serial Monitor.

D-pad and Buttons

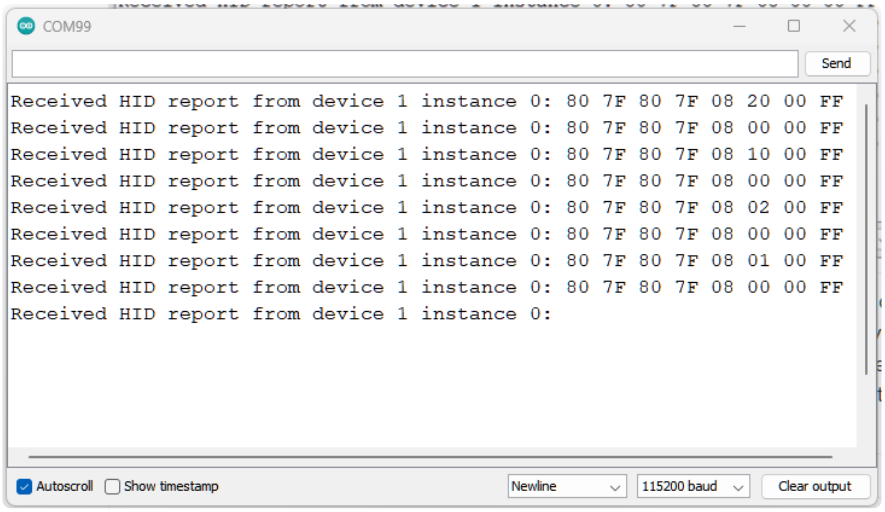


In the printout above, the A, B, Y and X buttons were pressed followed by up, down, left and right on the D-pad. All of these buttons' bytes are located on the fourth byte of the report.

You'll notice a repeating report with `0x08` in the fourth byte. This is the empty report, which is sent when buttons are released and no buttons are currently pressed. When a button is pressed, you'll see that the bitmask is added to the empty byte of `0x08`. For example, an A button prints out as `0x28`. This means that the A button byte is `0x20`, the B button prints out as `0x48`, so its byte is `0x40`, etc.

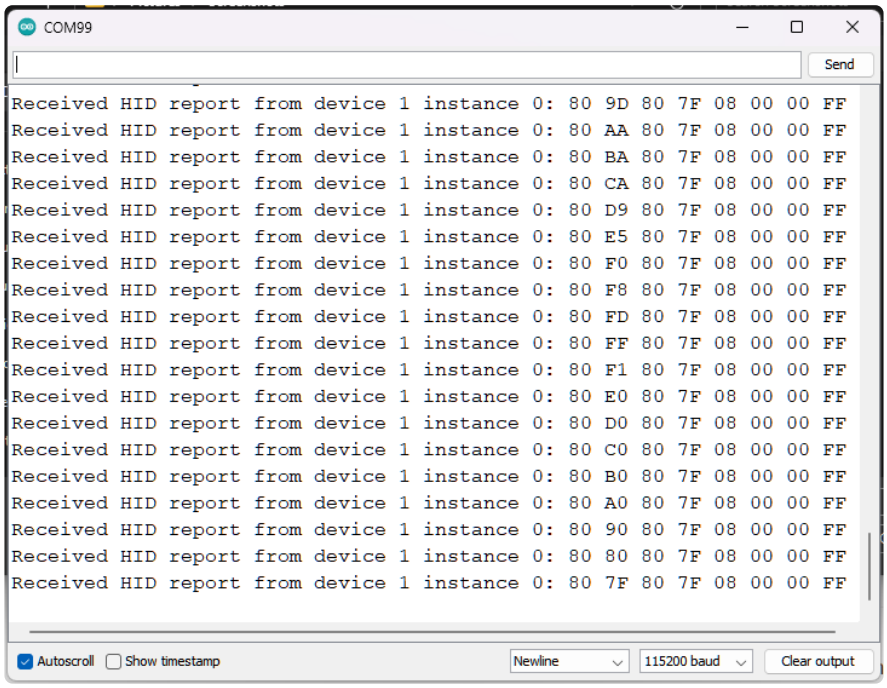
What about the D-pad? The empty neutral `0x08` actually means that no buttons on the D-pad are pressed. The D-pad buttons in this case start at `0x00` for up, `0x01` for up and right, `0x02` for right, etc.

Miscellaneous Buttons



This was followed by pressing the Start, Back, Right paddle and Left paddle buttons. You'll notice that these bytes are located on the fifth byte of the report, replacing `0x00` as the empty byte.

Analog Joysticks



The next inputs tested were the analog joysticks. Each joystick has an x and y axis. Each axis sends values to a different byte. In this case, the left x axis is on byte 0, the left y axis is on byte 1, the right x axis is on byte 2 and the right y axis is on byte 3. In the screenshot above, the left y axis is being moved. You'll notice that when an x axis

is in its default position, it returns a value of `0x80`. A y axis in default position returns a value of `0x7F`.

Log Your Reports

For the Arduino code, you'll need to include the HID reports for each of your gamepad buttons. Similar to the tests in the screenshots above, you'll want to go through and press each button on your gamepad to confirm its byte and byte address location. You'll need to edit the **gamepad_report.h** file:

```
// SPDX-FileCopyrightText: 2024 Liz Clark for Adafruit Industries
//
// SPDX-License-Identifier: MIT

// HID reports for Logitech Gamepad F310
// Update defines and combo_report for your gamepad and/or combo!

// Byte indices for the gamepad report
#define BYTE_LEFT_STICK_X    0   // Left analog stick X-axis
#define BYTE_LEFT_STICK_Y    1   // Left analog stick Y-axis
#define BYTE_RIGHT_STICK_X   2   // Right analog stick X-axis
#define BYTE_RIGHT_STICK_Y   3   // Right analog stick Y-axis
#define BYTE_DPAD_BUTTONS    4   // D-Pad and face buttons
#define BYTE_MISC_BUTTONS    5   // Miscellaneous buttons (triggers, paddles,
start, back)
#define BYTE_UNUSED          6   // Unused
#define BYTE_STATUS          7   // Status byte (usually constant)

// Button masks for Byte[4] (DPAD and face buttons)
#define DPAD_MASK            0x07  // Bits 0-2 for D-Pad direction
#define DPAD_NEUTRAL         0x08  // Bit 3 set when D-Pad is neutral

// D-Pad directions (use when DPAD_NEUTRAL is not set)
#define DPAD_UP              0x00  // 0000
#define DPAD_UP_RIGHT        0x01  // 0001
#define DPAD_RIGHT           0x02  // 0010
#define DPAD_DOWN_RIGHT      0x03  // 0011
#define DPAD_DOWN            0x04  // 0100
#define DPAD_DOWN_LEFT       0x05  // 0101
#define DPAD_LEFT            0x06  // 0110
#define DPAD_UP_LEFT         0x07  // 0111

// Face buttons (Byte[4] bits 4-7)
#define BUTTON_X             0x10
#define BUTTON_A             0x20
#define BUTTON_B             0x40
#define BUTTON_Y             0x80

// Button masks for Byte[5] (MISC buttons)
#define MISC_NEUTRAL         0x00

// Miscellaneous buttons (Byte[5])
#define BUTTON_LEFT_PADDLE   0x01
#define BUTTON_RIGHT_PADDLE  0x02
#define BUTTON_LEFT_TRIGGER  0x04
#define BUTTON_RIGHT_TRIGGER 0x08
#define BUTTON_BACK          0x10
#define BUTTON_START         0x20

#define LEFT_STICK_X_NEUTRAL   0x80
#define LEFT_STICK_Y_NEUTRAL   0x7F
#define RIGHT_STICK_X_NEUTRAL   0x80
#define RIGHT_STICK_Y_NEUTRAL   0x7F
```

```
uint8_t combo_report[] = { BUTTON_A, BUTTON_LEFT_PADDLE, BUTTON_RIGHT_PADDLE };
size_t combo_size = sizeof(combo_report) / sizeof(combo_report[0]);
```

Each button bitmask is included as a `#define`. Use the same variables and update the bitmasks for your controller. At the bottom of the header file, you'll see the `combo_report[]`. You'll want to include a list of the buttons you want to use as your combo. By default, the combo will trigger when an A button, left shoulder and right shoulder buttons are pressed together.

```
uint8_t combo_report[] = { BUTTON_A, BUTTON_LEFT_PADDLE, BUTTON_RIGHT_PADDLE };
```

More About Device Reports

If you think you'll be reading device reports often, check out the HID Reporter project (https://adafru.it/1a8x).

HID Reporter
By John Park
Overview

https://learn.adafruit.com/hid-reporter/overview

Arduino IDE Setup

The **Arduino Philhower core** (https://adafru.it/ToC) provides support for RP2040 microcontroller boards. This page covers getting your Arduino IDE set up to include your board.

Arduino IDE Download

The first thing you will need to do is to download the latest release of the Arduino IDE. The Philhower core requires **version 1.8** or higher.

Arduino IDE Download

https://adafru.it/Pd5

Download and install it to your computer.

Once installed, open the Arduino IDE.

Adding the Philhower Board Manager URL

In the Arduino IDE, and navigate to the **Preferences** window. You can access it through **File** > **Preferences** on Windows or Linux, or **Arduino** > **Preferences** on OS X.

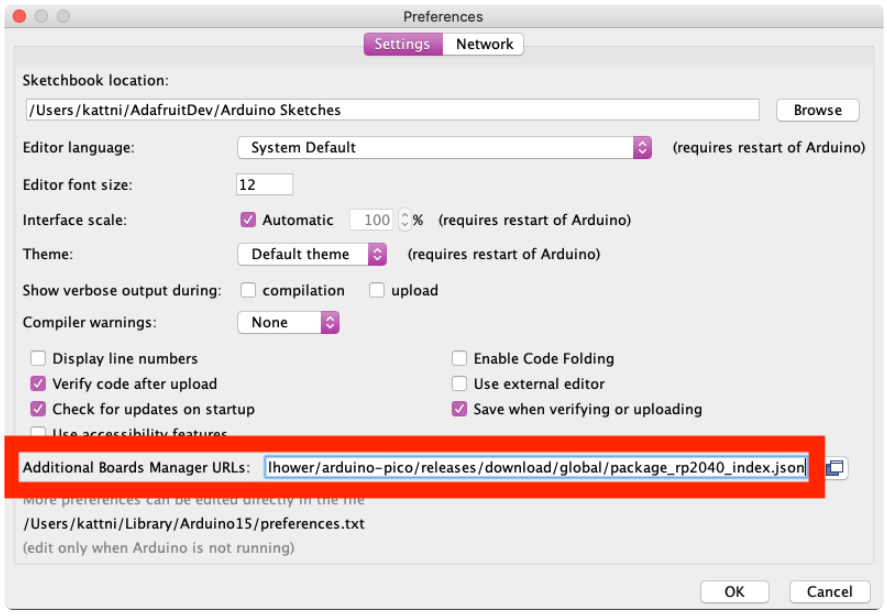
The **Preferences** window will open.

In the **Additional Boards Manager URLs** field, you'll want to add a new URL. The list of URLs is comma separated, and you will only have to add each URL once. The URLs point to index files that the Board Manager uses to build the list of available & installed boards.

Copy the following URL.

```
https://github.com/earlephilhower/arduino-pico/releases/download/global/package_rp2040_index.json
```

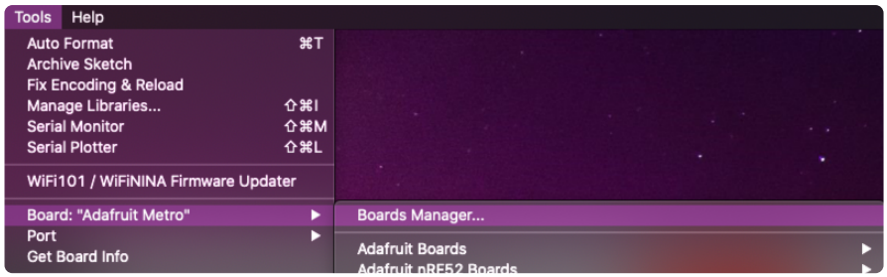
Add the URL to the the **Additional Boards Manager URLs** field (highlighted in red below).



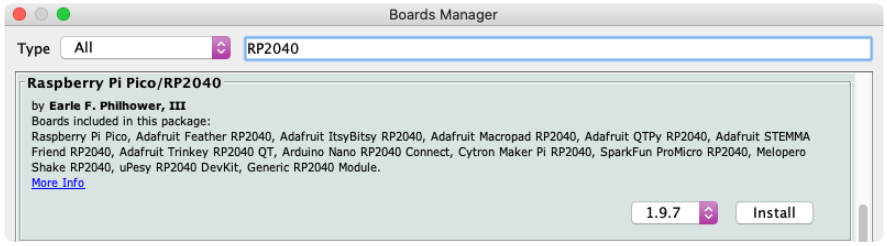
Click **OK** to save and close **Preferences**.

Add Board Support Package

In the Arduino IDE, click on **Tools** > **Board** > **Boards Manager**. If you have previously selected a board, the **Board** menu item may have a board name after it.



In the **Boards Manager**, search for RP2040. Scroll down to the **Raspberry Pi Pico/RP2040 by Earle F Philhower, III** entry. Click **Install** to install it.

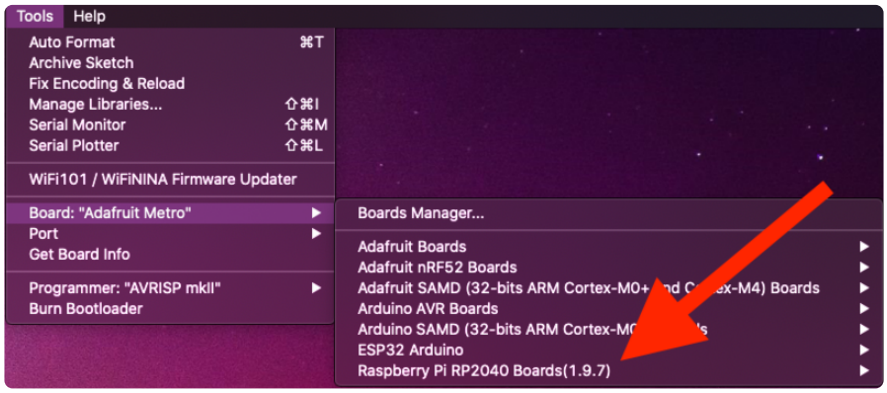


> Installing a new board package can take a few minutes. Don't click Cancel!

Once installation is complete, click **Close** to close the Boards Manager.

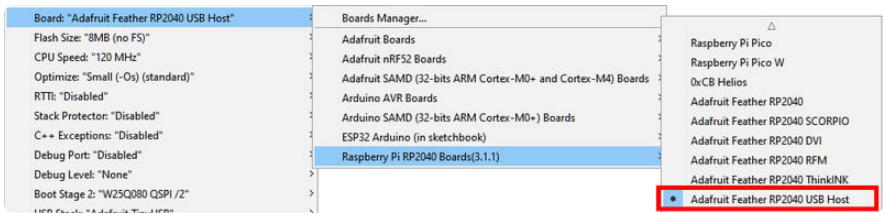
Choose Your Board

In the **Tools** > **Boards** menu, you should now see **Raspberry Pi RP2040 Boards** (possibly followed by a version number).



Navigate to the **Raspberry Pi RP2040 Boards** menu. You will see the available boards listed.

Navigate to the **Raspberry Pi RP2040 Boards** menu and choose **Adafruit Feather RP2040 USB Host**.



If there is no serial Port available in the dropdown, or an invalid one appears - don't worry about it! The RP2040 does not actually use a serial port to upload, so its OK if it does not appear if in manual bootload mode. You will see a serial port appear after uploading your first sketch.

Now you're ready to begin using Arduino with your RP2040 board!

Troubleshooting

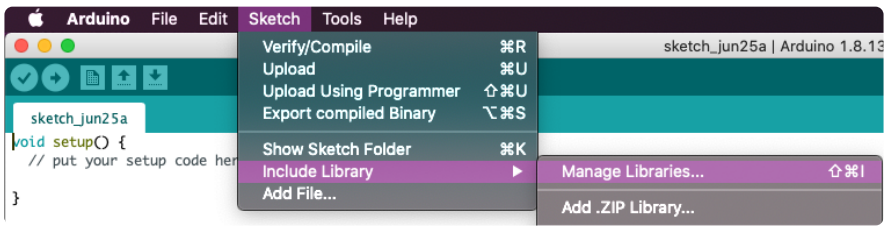
If you have any strange errors after updating the **Raspberry Pi Pico/RP2040 by Earle F Philhower, III** board support package (BSP) from the boards manager you may need to start with a fresh install of the BSP. Close out of the Arduino IDE and navigate to your Arduino packages folder: **C:\Users\[username]\AppData\Local\Arduino15\packages** and delete the **/rp2040** folder. After that, open the Arduino IDE and follow the steps above for installing the BSP. The errors should not occur.

Turbo Button Code

This project takes advantage of the fantastic TinyUSB Arduino library (https://adafru.it/EWc), which has both USB host and HID support. You'll need to install the necessary libraries and update the gamepad_report.h (https://adafru.it/1a8y) file as described on the previous page in this guide (https://adafru.it/1a8z) before uploading the code to your Feather RP2040 USB Host with the Arduino IDE.

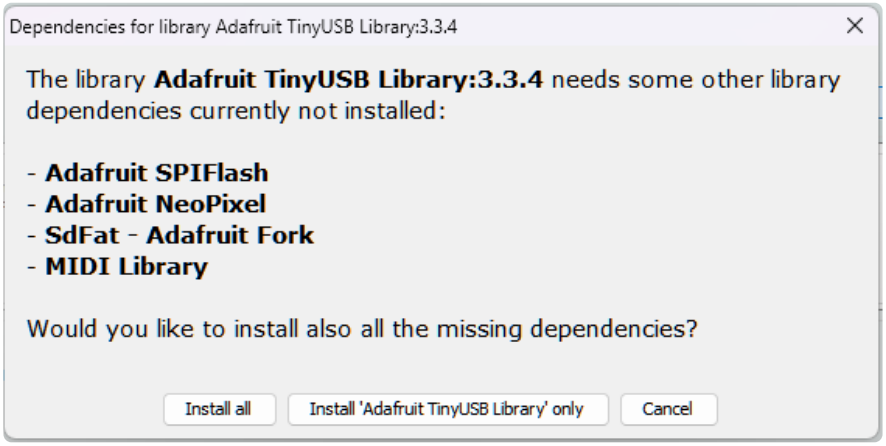
Install the Libraries

You can install the libraries for this project using the Library Manager in the Arduino IDE.

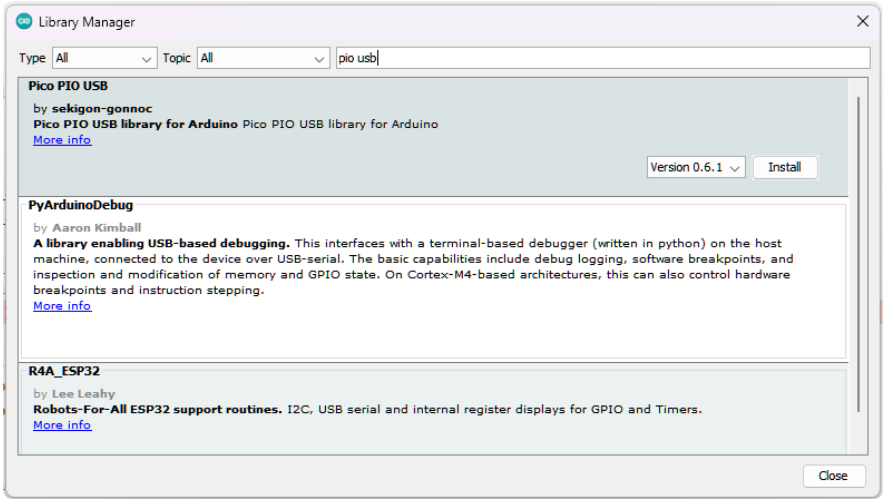


Click the **Manage Libraries...** menu item, search for **Adafruit TinyUSB Arduino**, and select the **Adafruit TinyUSB Library**:

If asked about dependencies, click "Install all".



Then install the Pico PIO USB library. Click the **Manage Libraries...** menu item again, search for **PIO USB**, and select the **Pico PIO USB** library by sekigon-gonnoc:



Code Prep

The code consists of a main **.ino** program file and two header files. The header files store configuration for USB host on the RP2040 and HID device reports for your gamepad. You'll need all three of these files to properly compile and run the project. These files are available in the .ZIP folder below or on GitHub (https://adafru.it/1a8A).

Arduino-USB-Host-Turbo-Button-Gamepad.zip

https://adafru.it/1a8B

```
// SPDX-FileCopyrightText: 2024 Liz Clark for Adafruit Industries
//
// SPDX-License-Identifier: MIT

/*********************************************************************
 Adafruit invests time and resources providing this open source code,
 please support Adafruit and open-source hardware by purchasing
 products from Adafruit!

 MIT license, check LICENSE for more information
 Copyright (c) 2019 Ha Thach for Adafruit Industries
 All text above, and the splash screen below must be included in
 any redistribution
*********************************************************************/

/* This example demonstrates use of both device and host, where
 * - Device runs on native USB controller (roothub port0)
 * - Host depends on MCU:
 *   - rp2040: bit-banging 2 GPIOs with Pico-PIO-USB library (roothub port1)
 *
 * Requirements:
 * - For rp2040:
 *   - Pico-PIO-USB library
 *   - 2 consecutive GPIOs: D+ is defined by PIN_USB_HOST_DP, D- = D+ +1
 *   - Provide VBus (5v) and GND for peripheral
 *   - CPU Speed must be either 120 or 240 MHz. Selected via "Menu -> CPU Speed"
 */

// USBHost is defined in usbh_helper.h
#include "usbh_helper.h"
#include "tusb.h"
#include "Adafruit_TinyUSB.h"
#include "gamepad_reports.h"

// HID report descriptor using TinyUSB's template
// Single Report (no ID) descriptor
uint8_t const desc_hid_report[] = {
    TUD_HID_REPORT_DESC_GAMEPAD()
};

// USB HID object
Adafruit_USBD_HID usb_hid;

// Report payload defined in src/class/hid/hid.h
// - For Gamepad Button Bit Mask see  hid_gamepad_button_bm_t
// - For Gamepad Hat    Bit Mask see  hid_gamepad_hat_t
hid_gamepad_report_t gp;

bool combo_active = false;

void setup() {
```

```
  if (!TinyUSBDevice.isInitialized()) {
    TinyUSBDevice.begin(0);
  }
  Serial.begin(115200);
  // Setup HID
  usb_hid.setPollInterval(2);
  usb_hid.setReportDescriptor(desc_hid_report, sizeof(desc_hid_report));
  usb_hid.begin();

  // If already enumerated, additional class driverr begin() e.g msc, hid, midi
won't take effect until re-enumeration
  if (TinyUSBDevice.mounted()) {
    TinyUSBDevice.detach();
    delay(10);
    TinyUSBDevice.attach();
  }
}

#if defined(ARDUINO_ARCH_RP2040)
//--------------------------------------------------------------------+
// For RP2040 use both core0 for device stack, core1 for host stack
//--------------------------------------------------------------------//

//------------- Core0 -------------//
void loop() {
}

//------------- Core1 -------------//
void setup1() {
  // configure pio-usb: defined in usbh_helper.h
  rp2040_configure_pio_usb();

  // run host stack on controller (rhport) 1
  // Note: For rp2040 pico-pio-usb, calling USBHost.begin() on core1 will have most
of the
  // host bit-banging processing works done in core1 to free up core0 for other
works
  USBHost.begin(1);
}

void loop1() {
  USBHost.task();
  Serial.flush();
  if (combo_active) {
    turbo_button();
  }
}
#endif

//--------------------------------------------------------------------+
// HID Host Callback Functions
//--------------------------------------------------------------------+

void tuh_hid_mount_cb(uint8_t dev_addr, uint8_t instance, uint8_t const*
desc_report, uint16_t desc_len)
{
  Serial.printf("HID device mounted (address %d, instance %d)\n", dev_addr,
instance);

  // Start receiving HID reports
  if (!tuh_hid_receive_report(dev_addr, instance))
  {
    Serial.printf("Error: cannot request to receive report\n");
  }
}

void tuh_hid_umount_cb(uint8_t dev_addr, uint8_t instance)
{
  Serial.printf("HID device unmounted (address %d, instance %d)\n", dev_addr,
```

```
instance);
}

void turbo_button() {
  if (combo_active) {
    while (!usb_hid.ready()) {
        yield();
      }
    Serial.println("A");
    gp.buttons = GAMEPAD_BUTTON_A;
    usb_hid.sendReport(0, &gp, sizeof(gp));
    Serial.println("off");
    delay(2);
    gp.buttons = 0;
    usb_hid.sendReport(0, &gp, sizeof(gp));
    delay(2);
  }
}

bool is_combo_detected(const uint8_t* combo, size_t combo_size, const uint8_t*
report) {
  for (size_t i = 0; i < combo_size; i++) {
    uint8_t button = combo[i];

    // Determine which byte the button is in and then check if it is pressed
    if (button & BUTTON_A || button & BUTTON_B || button & BUTTON_X || button &
BUTTON_Y) {
      if ((report[BYTE_DPAD_BUTTONS] & button) != button) {
        return false;  // Button is not pressed
      }
    } else if (button & BUTTON_LEFT_PADDLE || button & BUTTON_RIGHT_PADDLE ||
               button & BUTTON_LEFT_TRIGGER || button & BUTTON_RIGHT_TRIGGER ||
               button & BUTTON_BACK || button & BUTTON_START) {
      if ((report[BYTE_MISC_BUTTONS] & button) != button) {
        return false;  // Button is not pressed
      }
    }
  }
  return true;  // All buttons in the combo are pressed
}

void tuh_hid_report_received_cb(uint8_t dev_addr, uint8_t instance, uint8_t const*
report, uint16_t len) {

  // Manage the combo state and print messages
  if ((is_combo_detected(combo_report, combo_size, report)) && !combo_active) {
    combo_active = true;
    Serial.println("combo!");
  } else if ((is_combo_detected(combo_report, combo_size, report)) && combo_active)
{
    combo_active = false;
    Serial.println("combo released!");
  }
    // Handle analog stick inputs
    gp.x = (report[BYTE_LEFT_STICK_X] != LEFT_STICK_X_NEUTRAL) ?
map(report[BYTE_LEFT_STICK_X], 0, 255, -127, 127) : 0;
    gp.y = (report[BYTE_LEFT_STICK_Y] != LEFT_STICK_Y_NEUTRAL) ?
map(report[BYTE_LEFT_STICK_Y], 0, 255, -127, 127) : 0;
    gp.z = (report[BYTE_RIGHT_STICK_X] != RIGHT_STICK_X_NEUTRAL) ?
map(report[BYTE_RIGHT_STICK_X], 0, 255, -127, 127) : 0;
    gp.rz = (report[BYTE_RIGHT_STICK_Y] != RIGHT_STICK_Y_NEUTRAL) ?
map(report[BYTE_RIGHT_STICK_Y], 0, 255, -127, 127) : 0;

    // Debug print for report data
    /*Serial.print("Report data: ");
    for (int i = 0; i < len; i++) {
      Serial.print(report[i], HEX);
      Serial.print(" ");
    }
```

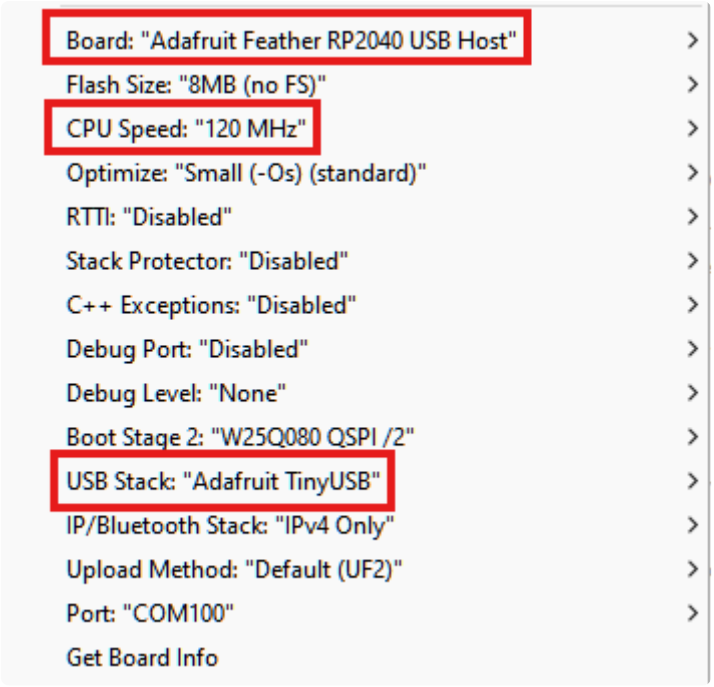
```
    Serial.println();*/

    uint8_t dpad_value = report[BYTE_DPAD_BUTTONS] & DPAD_MASK;

    // Handle D-pad direction
    if ((report[BYTE_DPAD_BUTTONS] & DPAD_NEUTRAL) == 0) {
      switch (dpad_value) {
        case DPAD_UP: gp.hat = 1; Serial.println("up"); break;
        case DPAD_UP_RIGHT: gp.hat = 2; Serial.println("up/right"); break;
        case DPAD_RIGHT: gp.hat = 3; Serial.println("right"); break;
        case DPAD_DOWN_RIGHT: gp.hat = 4; Serial.println("down/right"); break;
        case DPAD_DOWN: gp.hat = 5; Serial.println("down"); break;
        case DPAD_DOWN_LEFT: gp.hat = 6; Serial.println("down/left"); break;
        case DPAD_LEFT: gp.hat = 7; Serial.println("left"); break;
        case DPAD_UP_LEFT: gp.hat = 8; Serial.println("up/left"); break;
        }
    } else {
      gp.hat = 0;
    }
    uint16_t buttons = gp.buttons;
    if ((report[BYTE_DPAD_BUTTONS] & BUTTON_X) == BUTTON_X) {
      buttons |= GAMEPAD_BUTTON_X;
      Serial.println("x");
    } else {
      buttons &= ~GAMEPAD_BUTTON_X;
    }
    if ((report[BYTE_DPAD_BUTTONS] & BUTTON_A) == BUTTON_A) {
      buttons |= GAMEPAD_BUTTON_A;
      Serial.println("a");
    } else {
      buttons &= ~GAMEPAD_BUTTON_A;
    }
    if ((report[BYTE_DPAD_BUTTONS] & BUTTON_B) == BUTTON_B) {
      buttons |= GAMEPAD_BUTTON_B;
      Serial.println("b");
    } else {
      buttons &= ~GAMEPAD_BUTTON_B;
    }
    if ((report[BYTE_DPAD_BUTTONS] & BUTTON_Y) == BUTTON_Y) {
      buttons |= GAMEPAD_BUTTON_Y;
      Serial.println("y");
    } else {
      buttons &= ~GAMEPAD_BUTTON_Y;
    }
    gp.buttons = buttons;

    if ((report[BYTE_MISC_BUTTONS] & BUTTON_LEFT_PADDLE) == BUTTON_LEFT_PADDLE) {
      buttons |= GAMEPAD_BUTTON_TL;
      Serial.println("left paddle");
    } else {
      buttons &= ~GAMEPAD_BUTTON_TL;
    }
    if ((report[BYTE_MISC_BUTTONS] & BUTTON_RIGHT_PADDLE) == BUTTON_RIGHT_PADDLE) {
      buttons |= GAMEPAD_BUTTON_TR;
      Serial.println("right paddle");
    } else {
      buttons &= ~GAMEPAD_BUTTON_TR;
    }
    if ((report[BYTE_MISC_BUTTONS] & BUTTON_LEFT_TRIGGER) == BUTTON_LEFT_TRIGGER) {
      buttons |= GAMEPAD_BUTTON_TL2;
      Serial.println("left trigger");
    } else {
      buttons &= ~GAMEPAD_BUTTON_TL2;
    }
    if ((report[BYTE_MISC_BUTTONS] & BUTTON_RIGHT_TRIGGER) == BUTTON_RIGHT_TRIGGER)
{
      buttons |= GAMEPAD_BUTTON_TR2;
      Serial.println("right trigger");
    } else {
```

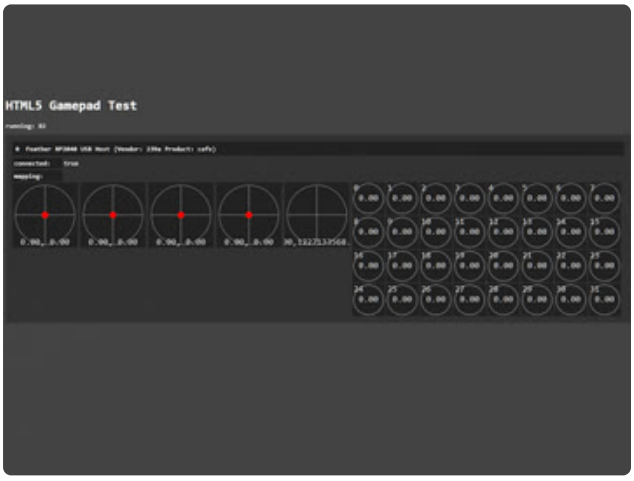
```
      buttons &= ~GAMEPAD_BUTTON_TR2;
    }
    if ((report[BYTE_MISC_BUTTONS] & BUTTON_BACK) == BUTTON_BACK) {
      buttons |= GAMEPAD_BUTTON_SELECT;
      Serial.println("back");
    } else {
      buttons &= ~GAMEPAD_BUTTON_SELECT;
    }
    if ((report[BYTE_MISC_BUTTONS] & BUTTON_START) == BUTTON_START) {
      buttons |= GAMEPAD_BUTTON_START;
      Serial.println("start");
    } else {
      buttons &= ~GAMEPAD_BUTTON_START;
    }
    // Set the final buttons state
    gp.buttons = buttons;

  // Debug print for gp contents
  /*Serial.print("gp.x: "); Serial.println(gp.x);
  Serial.print("gp.y: "); Serial.println(gp.y);
  Serial.print("gp.z: "); Serial.println(gp.z);
  Serial.print("gp.rz: "); Serial.println(gp.rz);
  Serial.print("gp.hat: "); Serial.println(gp.hat);
  Serial.print("gp.buttons: "); Serial.println(gp.buttons, HEX);*/

  while (!usb_hid.ready()) {
    yield();
  }
  usb_hid.sendReport(0, &gp, sizeof(gp));

  // Continue to receive the next report
  if (!tuh_hid_receive_report(dev_addr, instance)) {
    Serial.println("Error: cannot request to receive report");
  }
}
```

After downloading the files and opening them in the Arduino IDE, navigate to the **gamepad_reports.h** header file. Update the file to match your gamepad information as described in the Read Your Gamepad Device Reports (https://adafru.it/1a8z) page.

Upload and Test

Before uploading, you'll need to update some settings in the Boards menu. Select the **Adafruit Feather RP2040 USB Host** as your board. Under **CPU Speed**, select **120 MHz**. Under **USB Stack** select **Adafruit TinyUSB**. Then, upload the sketch to your board. You can use the Serial Monitor in the Arduino IDE for debugging any errors.





You can test your gamepad by using a browser-based gamepad tester. This gamepad tester by greggman (https://adafru.it/1a8C) is hosted on GitHub pages and uses HTML. Test out all of the buttons and joysticks that you setup in your **gamepad_reports.h** file to make sure they are being recognized and responding as expected.

How the Code Works

The code takes advantage of the two cores on the RP2040. The HID device code runs on core 0 and the USB host code runs on core 1. The HID device code enumerates as a USB gamepad. This means that the Feather will show up as an HID input device when its plugged in over USB.

```
//------------- Core0 -------------//
void setup() {
  if (!TinyUSBDevice.isInitialized()) {
    TinyUSBDevice.begin(0);
  }
  Serial.begin(115200);
  // Setup HID
  usb_hid.setPollInterval(2);
  usb_hid.setReportDescriptor(desc_hid_report, sizeof(desc_hid_report));
  usb_hid.begin();

  // If already enumerated, additional class driverr begin() e.g msc, hid, midi
won't take effect until re-enumeration
  if (TinyUSBDevice.mounted()) {
```

```
    TinyUSBDevice.detach();
    delay(10);
    TinyUSBDevice.attach();
  }
}

//------------- Core1 -------------//
void setup1() {
  // configure pio-usb: defined in usbh_helper.h
  rp2040_configure_pio_usb();

  // run host stack on controller (rhport) 1
  // Note: For rp2040 pico-pio-usb, calling USBHost.begin() on core1 will have most
of the
  // host bit-banging processing works done in core1 to free up core0 for other
works
  USBHost.begin(1);
}
```

Nothing is run in the core 0 loop. The core 1 loop runs the USB host callback functions.

```
void loop1() {
  USBHost.task();
  Serial.flush();
  if (combo_active) {
    turbo_button();
  }
}
```

The callback functions include mounting the device and unmounting the device. The main function that detects the incoming button presses is the `tuh_hid_report_received_cb()`. If an input is received, the incoming device report is compared to the defined bitmasks in the **gamepad_reports.h** file. If a button matches, then that button press is sent; essentially making the Feather act as a passthrough for the attached gamepad.

```
uint8_t dpad_value = report[BYTE_DPAD_BUTTONS] &amp;amp; DPAD_MASK;

    // Handle D-pad direction
    if ((report[BYTE_DPAD_BUTTONS] &amp;amp; DPAD_NEUTRAL) == 0) {
      switch (dpad_value) {
        case DPAD_UP: gp.hat = 1; Serial.println("up"); break;
        case DPAD_UP_RIGHT: gp.hat = 2; Serial.println("up/right"); break;
        case DPAD_RIGHT: gp.hat = 3; Serial.println("right"); break;
        case DPAD_DOWN_RIGHT: gp.hat = 4; Serial.println("down/right"); break;
        case DPAD_DOWN: gp.hat = 5; Serial.println("down"); break;
        case DPAD_DOWN_LEFT: gp.hat = 6; Serial.println("down/left"); break;
        case DPAD_LEFT: gp.hat = 7; Serial.println("left"); break;
        case DPAD_UP_LEFT: gp.hat = 8; Serial.println("up/left"); break;
        }
    } else {
      gp.hat = 0;
    }
...
while (!usb_hid.ready()) {
    yield();
  }
  usb_hid.sendReport(0, &amp;amp;gp, sizeof(gp));
```

The combo detection for a turbo button is handled with the boolean `is_combo_detected()`. This checks for a match with the `combo_report[]` that you defined in the **gamepad_report.h** file. If a combo is detected, then `combo_active` is toggled.

```
if ((is_combo_detected(combo_report, combo_size, report)) &amp;amp;&amp;amp; !combo_active)
{
    combo_active = true;
    Serial.println("combo!");
  } else if ((is_combo_detected(combo_report, combo_size, report)) &amp;amp;&amp;amp;
combo_active) {
    combo_active = false;
    Serial.println("combo released!");
  }
```

The `turbo_button()` function is run while `combo_active` is `true`. It sends the A button rapidly via the HID device. You can edit this function to send a variety of inputs depending on your needs.

```
void turbo_button() {
  if (combo_active) {
    while (!usb_hid.ready()) {
        yield();
      }
    Serial.println("A");
    gp.buttons = GAMEPAD_BUTTON_A;
    usb_hid.sendReport(0, &amp;gp, sizeof(gp));
    Serial.println("off");
    delay(2);
    gp.buttons = 0;
    usb_hid.sendReport(0, &amp;gp, sizeof(gp));
    delay(2);
  }
}
```

Assembly

You can house the Feather RP2040 USB Host inside the Snap-on Enclosure for Adafruit Feather RP2040 USB Host (http://adafru.it/6057).

Snap-on Enclosure for Adafruit Feather RP2040 USB Host
Here is a cute and minimal enclosure for your Adafruit Feather RP2040 with USB Type A Host to keep it safe during use and...
https://www.adafruit.com/product/6057

Insert the Feather into the bottom of the case, lining up the mounting pegs with the mounting holes on the board.

Snap-on the lid, lining up the cutouts for the boot and reset buttons. You can also use the cutout for the STEMMA QT connector as a guide.

Now your Feather is safe and sound inside its enclosure and ready to act as a USB host!

Use

Plug in your Feather to your USB gaming device, be it a PC or gaming system. Then, plug in your gamepad to the Feather.

You may need to calibrate your controller, depending on what you're using it with. For example, Steam requires you to setup any custom controllers within Settings. You can also test it ahead of time with a browser-based gamepad tester (https://adafru.it/1a8C).

> And again, this guide is meant to demonstrate what is possible with USB host and should be used as an experiment or in aiding accessible gaming. This isn't a walkthrough on how to cheat in video games, especially in online or competitive play.

Once you're playing your game, you can try out your combo to turn on turbo mode! The mode is toggled with the combo, so just press the combo again to stop it.

This project can be modified to be more than just a turbo button. You could change the code to send a specific button combo (up, up, down, down, left, right..), listen for multiple combos or even add an external foot pedal or other accessible controller to send certain button presses. If gaming isn't your thing, you could also experiment with changing the code to act as a different type of HID device, like a MIDI controller, and have the gamepad button combinations trigger certain MIDI events.